

Internship Final Report

Student Name: Kunda kiranmai

University: Adikavi Nannayya University

Major: M.sc(Cybersecurity)

Internship Duration: April 10th, 2025 - May 3rd, 2025

Company: Hack Secure

Domain: Cyber Security

Mentor: Mr.Nishant Prajapati

Assistant Mentor: Mr. Aman Pandey

Coordinator: Mr. Shivam Kapoor

Objectives :

My primary objectives for this internship were to:

1. Develop a deep understanding of cybersecurity principles and practices.
2. Gain hands-on experience in identifying, analyzing, and mitigating security threats.
3. Enhance my skills in using cybersecurity tools and techniques in real-world scenarios.

Tasks and Responsibilities :

During my internship, I was involved in the following key tasks:

- **Vulnerability Assessment:** Conducted a thorough scan of a target website to identify open ports and potential vulnerabilities.
- **Penetration Testing:** Performed brute-force attacks and directory enumeration on the website to uncover hidden directories and files.
- **Traffic Analysis:** Intercepted network traffic using Wireshark during a simulated login attempt, successfully capturing and analyzing transmitted credentials.
- **Performed Owasp top 10 Attacks:** I have Performed SQL injection on the login or search page of <http://testphp.vulnweb.com> and checked if the website is vulnerable to SQLi by extracting database information and also Injected malicious JavaScript payloads in input fields (such as the comment section or search box) to see if the website is vulnerable to stored or reflected XSS attacks.
- **Port scanning by python script:** Used Python script to scan open ports on a given IP address and port range.

Intermediate Task

Task 1 – Nmap(Open ports scanning)

***Outcomes –**

Summary:

As part of the vulnerability assessment, I performed a reconnaissance scan on the domain test.vulnweb.com using the Nmap tool. Nmap (Network Mapper) is a powerful open-source utility used for network discovery and security auditing. It helps in identifying open ports, services running on those ports, and potential vulnerabilities associated with them.

***Conclusion -**

Skills Gained:

- Acquired hands-on experience with **Nmap**, one of the most widely used tools in cybersecurity for port scanning and service enumeration.
- Understood how to interpret open port results and relate them to potential security risks.
- Learned how to construct effective Nmap commands for both basic and service version detection scans.

Challenges Faced:

- Initially, I faced difficulty in understanding the different scan types and flags in Nmap (e.g., `-sS`, `-sV`, `-A`).
- It was challenging to interpret ambiguous or filtered port results and understand what services might be running behind them.

Improvements:

- I overcame the tool usage challenge by referring to the official Nmap documentation and practicing various scan types in a virtual lab environment.
- My ability to assess risk based on port scan results improved significantly.
- I also began correlating open ports with possible attack vectors, laying the groundwork for more advanced assessments such as vulnerability scanning and exploitation.

***Screenshot -**

The screenshot displays the output of an Nmap scan performed on the domain test.vulnweb.com using the `-sV` flag, which enables service version detection. The scan results indicate that **port 80/tcp is open** and hosting an **HTTP service**, typically associated with a web server. The presence of an open HTTP port suggests that the website is publicly accessible and could be further analyzed for web vulnerabilities such as outdated server software, misconfigured headers, or exposed directories. This initial enumeration step is crucial in ethical hacking, as it helps identify potential attack surfaces before moving on to more targeted assessments like vulnerability scanning or exploitation. The scan also confirms that the target is live and responsive, validating its suitability for further testing in a controlled ethical hacking environment.

```

root@kali:~/home/kali# nmap testphp.vulnweb.com -T5 -A
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-04-13 10:34 EDT
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.27s latency).
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx 1.19.0
|_http-title: Home of Acunetix Art
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
OS fingerprint not ideal because: Timing level 5 (Insane) used
No OS matches for host
Network Distance: 19 hops

TRACEROUTE (using port 80/tcp)
HOP RTT      ADDRESS
1  3.61 ms  192.168.1.1
2  4.87 ms  hyd-tdc-bngs-08 (110.235.231.66)
3  ...
4  4.05 ms  144.48.72.160
5  49.16 ms 10.240.248.100
6  23.20 ms 10.240.248.1
7  22.05 ms 14.142.71.93.static-hydrabad.vsnl.net.in (14.142.71.93)
8  75.68 ms 172.29.251.34
9  17.78 ms ix-ae-4-2.tcore2.cxr-chennai.as6453.net (180.87.37.1)
10 276.69 ms if-ae-3-3.tcore1.cxr-chennai.as6453.net (180.87.36.5)
11 278.72 ms if-bundle-26-2.qcore1.cxr-chennai.as6453.net (180.87.36.139)
12 ...
13 195.78 ms if-et-23-2.hcore2.kv8-chiba.as6453.net (180.87.67.33)
14 305.92 ms if-ae-5-3.tcore2.svl-santaclara.as6453.net (209.58.86.68)
15 ...
16 283.12 ms if-ae-20-3.tcore1.00s-seattle.as6453.net (63.243.225.54)
17 ... 18
19 246.05 ms ec2-44-228-249-3.us-west-2.compute.amazonaws.com (44.228.249.3)

```

Task 2 – Brute forcing the website.

*Outcomes -

Summary:

Using the **Kali Linux terminal**, I performed a directory brute-force attack on the website `http://testphp.vulnweb.com/` with the help of **Dirsearch**, a powerful command-line tool designed for web path discovery. Dirsearch works by sending multiple HTTP requests using a predefined wordlist to detect accessible but hidden directories on a website.

The command I used in the Kali terminal was

dirsearch -u http://testphp.vulnweb.com/

*Conclusion -

Skills Gained:

- Gained hands-on experience using **Dirsearch** in the **Kali Linux terminal** for directory enumeration.
- Learned how to perform **brute-force web path discovery** and interpret the results effectively.
- Developed a deeper understanding of how hidden directories can lead to further **web vulnerabilities** or unauthorized access points.

Challenges Faced:

- Initially faced difficulty in selecting the right wordlist and understanding status codes like 403 (Forbidden) or 301 (Redirect).
- The raw terminal output was overwhelming at first, especially distinguishing relevant results from false positives.
- Some directories were protected or redirected, requiring further validation through manual inspection.

Improvements:

- I improved my command-line skills in Kali Linux and learned to tweak Dirsearch options for better accuracy (e.g., using `--exclude-status`, `--threads`, or custom user agents).
- Became more confident in identifying which discovered paths could pose real security concerns.
- Now better equipped to use other reconnaissance tools in Kali Linux and integrate directory enumeration as a routine part of penetration testing.

***Screenshot -**

The screenshot captures the output of a directory brute-force scan performed using the Dirsearch tool in the Kali Linux terminal. The scan was executed against the target URL `http://testphp.vulnweb.com/`, revealing multiple accessible directories such as `/admin/`, `/uploads/`, `/images/`, and `/include/`. These directories are not typically visible to users through the web interface but can be discovered through forced browsing techniques. Identifying such directories is critical in the reconnaissance phase of ethical hacking, as they may expose sensitive resources or entry points that attackers could exploit. The use of Dirsearch through the command line not only strengthened my skills in web application enumeration but also enhanced my ability to interpret HTTP responses and focus on meaningful security findings.

```
[root@kali:~]# cd /home/kali
└─$ dirsearch -u http://testphp.vulnweb.com/
[08/16/2020:17:00:00] DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pyys.io/en/latest/pkg_resources.html
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

Extensions: php, asp, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 31460
Output File: /home/kali/reports/http_testphp_vulnweb.com/_29-06-13-10-36-17.txt

Target: http://testphp.vulnweb.com/

[10:36:17] Starting
[10:36:17] 301 → 309n ~ /.idea → http://testphp.vulnweb.com/.idea/
[10:36:17] 200 ~ 301d ~ /.idea/
[10:36:16 200 ~ 40 ~ /.idea/name
[10:36:16 200 ~ 131d ~ /.idea/settings.xml
[10:36:16 200 ~ 160d ~ /.idea/misc.xml
[10:36:16 200 ~ 232d ~ /.idea/modules.xml
[10:36:16 200 ~ 143d ~ /.idea/scopes/scope_settings.xml
[10:36:16 200 ~ 123d ~ /.idea/vcs.xml
[10:36:16 200 ~ 123d ~ /.idea/workspace.xml
[10:36:16 200 ~ 40d ~ .php
[10:36:16 200 ~ 400d ~ /_cms/serverScripts/
[10:36:16 200 ~ 160d ~ /_cms/serverScripts/showTSPG.php
[10:37:00 301 ~ 309n ~ admin → http://testphp.vulnweb.com/admin/
[10:37:00 200 ~ 262n ~ admin/
[10:37:00 200 ~ 340 ~ cgi-bin/
[10:37:34 403 ~ 276n ~ cgi-bin/
[10:37:34 403 ~ 276n ~ cgi-bin/
[10:37:35 403 ~ 273n ~ cgi-bin/status/index.cgi
[10:37:35 403 ~ 273n ~ cgi-bin/image.asp?72,2
[10:37:34 403 ~ 273n ~ cgi-bin/images.asp?72,2
[10:37:35 403 ~ 273n ~ cgi-bin/htmlscript
[10:37:35 403 ~ 273n ~ cgi-bin/pwstat.pl
[10:37:34 403 ~ 273n ~ cgi-bin/statstat.cgi
[10:37:34 403 ~ 273n ~ cgi-bin/login
[10:37:35 403 ~ 273n ~ cgi-bin/logout.xml
[10:37:34 403 ~ 273n ~ cgi-bin/login.cgi
[10:37:34 403 ~ 273n ~ cgi-bin/mf_welcome.cgi
[10:37:34 403 ~ 273n ~ cgi-bin/mf.cgi
[10:37:34 403 ~ 273n ~ cgi-bin/mf_mfwelcome.cgi
[10:37:34 403 ~ 273n ~ cgi-bin/mf7/mf_welcome.cgi
[10:37:34 403 ~ 273n ~ cgi-bin/mf7/mf.cgi
[10:37:34 403 ~ 273n ~ cgi-bin/mf7/mf.cgi
[10:37:34 403 ~ 273n ~ cgi-bin/printenv.pl
[10:37:34 403 ~ 273n ~ cgi-bin/printenv.pl
```

```
[10:37:34] 404 - 273B - /cgi-bin/mt/mt-xmlrpc.cgi
[10:37:34] 404 - 273B - /cgi-bin/mt7/mt-xmlrpc.cgi
[10:37:34] 404 - 273B - /cgi-bin/mt/mt.cgi
[10:37:34] 404 - 273B - /cgi-bin/mt7/mt.cgi
[10:37:34] 404 - 273B - /cgi-bin/printenv
[10:37:34] 404 - 273B - /cgi-bin/printenv.pl
[10:37:34] 404 - 273B - /cgi-bin/test.cgi
[10:37:34] 404 - 273B - /cgi-bin/test.cgi
[10:37:34] 404 - 273B - /cgi-bin/php.ini
[10:37:34] 404 - 273B - /cgi-bin/ViewLog.asp
[10:37:34] 404 - 273B - /cgi-bin2/
[10:37:42] 301 - 169B - /Connections → http://testphp.vulnweb.com/Connections/
[10:37:43] 200 - 224B - /crossdomain.xml
[10:37:44] 200 - 595B - /CVS/
[10:37:44] 200 - 1B - /CVS/Entries
[10:37:44] 200 - 1B - /CVS/Root
[10:38:00] 200 - 804B - /favicon.ico
[10:38:10] 301 - 169B - /images → http://testphp.vulnweb.com/images/
[10:38:10] 200 - 377B - /images/
[10:38:11] 200 - 3KB - /index.bak
[10:38:11] 200 - 3KB - /index.zip
[10:38:19] 200 - 5KB - /login.php
[10:38:20] 200 - 5KB - /logout.php
[10:38:38] 301 - 169B - /pictures → http://testphp.vulnweb.com/pictures/
[10:38:44] 200 - 5KB - /product.php
[10:38:52] 200 - 5KB - /search.php
[10:38:52] 301 - 169B - /secured → http://testphp.vulnweb.com/secured/
[10:38:57] 200 - 6KB - /signup.php
[10:39:15] 302 - 14B - /userinfo.php → login.php
[10:39:16] 200 - 268B - /vendor/
Task Completed
```

Task 3 - Intercepting the network traffic using Wireshark.

*Outcomes -

Summary:

To analyze the security of data transmitted over an unencrypted network, I used Wireshark—a powerful network protocol analyzer—on Kali Linux to capture HTTP traffic. I navigated to a test website that uses HTTP (not HTTPS), entered login credentials, and monitored the packet flow through Wireshark and inspecting packet contents, I successfully intercepted the **login request** and extracted the **username and password** sent in plaintext through the network. This exercise highlighted the risks associated with transmitting sensitive data over HTTP, which lacks encryption and is vulnerable to interception by anyone with access to the network.

*Conclusion –

Skills Gained:

- Learned to use Wireshark to capture and analyze real-time network traffic.
- Understood how HTTP-based login data can be intercepted due to the absence of encryption.
- Gained proficiency in using display filters to narrow down traffic and locate specific data such as POST requests and credentials.

Challenges Faced:

- Initially had difficulty identifying the correct packet containing the login credentials due to high traffic noise.
- Misinterpreted certain packets that were irrelevant to the login action, requiring multiple attempts and refined filtering.

- Understanding the raw structure of HTTP packets in the packet details pane took practice.

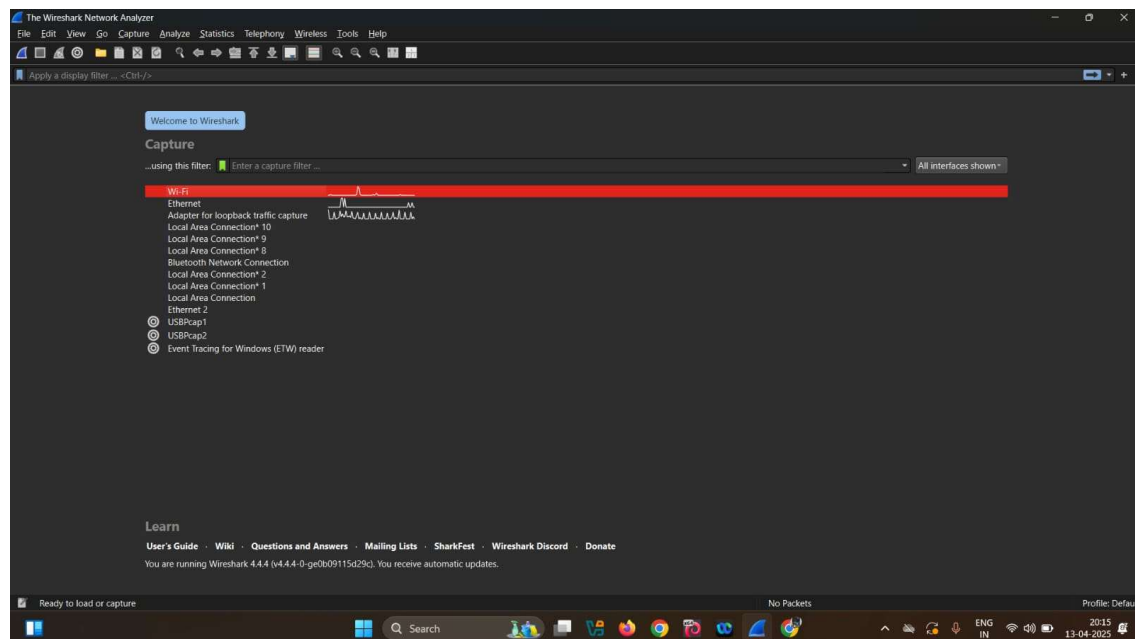
Improvements:

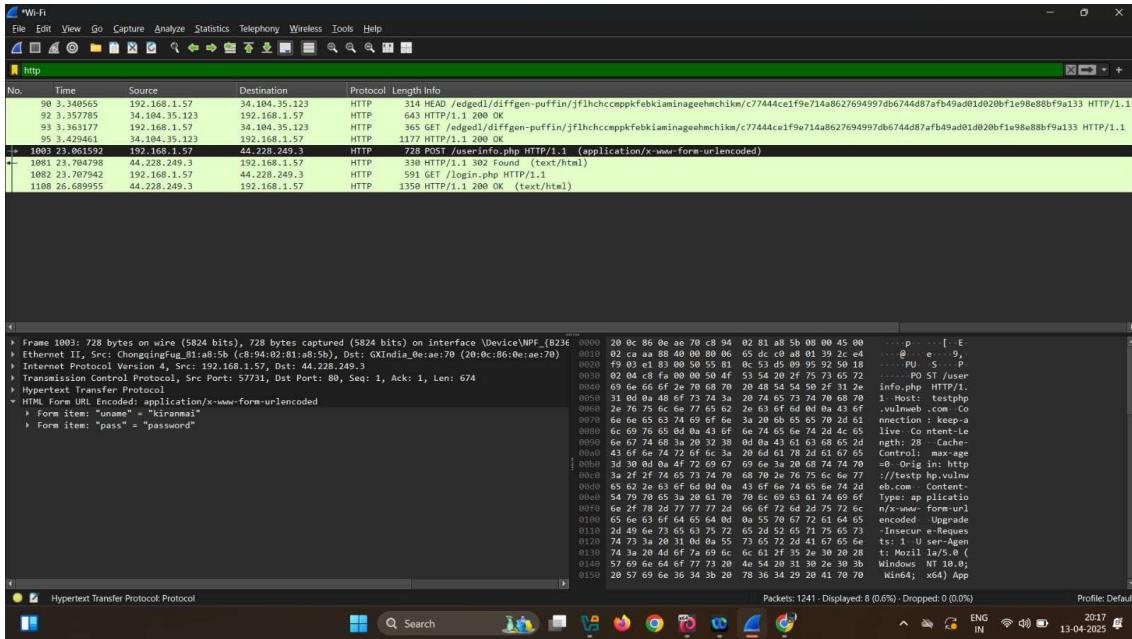
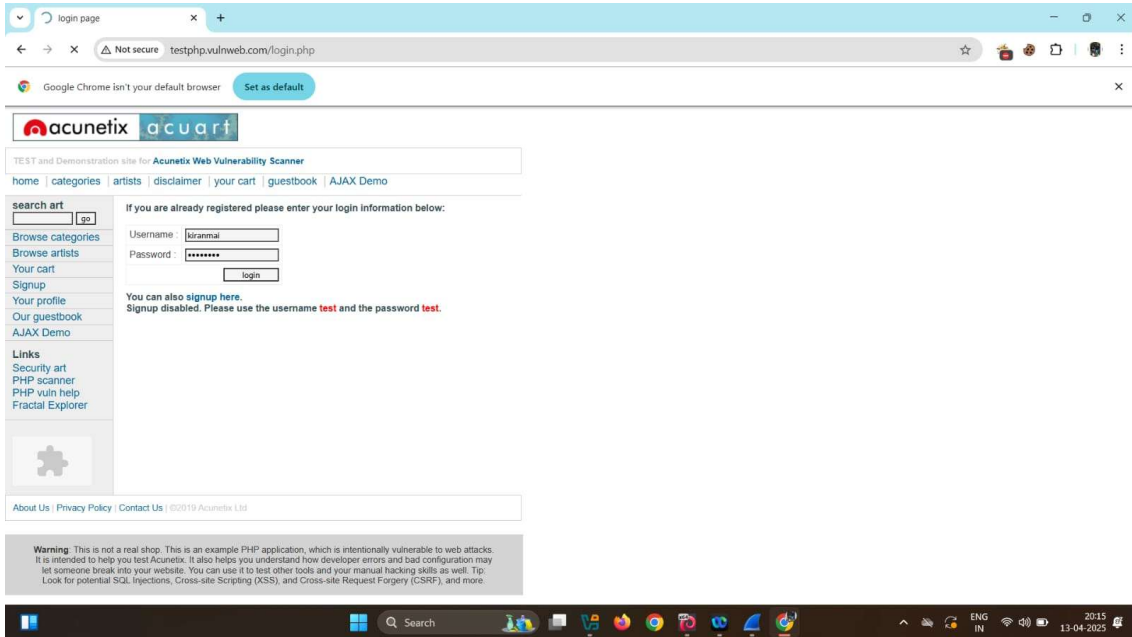
- Improved in applying effective display filters like `http` and `http.request.method` to isolate useful data.
- Learned to decode packet payloads and locate data within HTML form fields.
- Became more comfortable using Wireshark in various penetration testing scenarios to analyze insecure data transmission.

*Screenshot -

Title: Capturing HTTP Login Credentials Using Wireshark

The screenshot displays a captured HTTP POST request in Wireshark, showing login credentials transmitted in plaintext. The data was intercepted during a login attempt on a non-secure (HTTP) website. Using display filters, I was able to isolate the relevant packet and view the contents of the login form—including the entered username and password—in the packet payload. This activity demonstrates how easily credentials can be compromised on unsecured networks, reinforcing the importance of using HTTPS for all user authentication processes.





Task 4 - Performed SQL injection on the login page.

***Outcomes** -

Summary:

To test the SQL injection vulnerability on the target site <http://testphp.vulnweb.com/>, I attempted injecting SQL payloads into the login input fields. This technique exploits weak server-side input validation that allows attackers to manipulate backend SQL queries.

Using a basic SQL injection payload like:

' OR '1'='1 –

I submitted the login form, and the website granted unauthorized access, bypassing normal authentication. This confirms that the application is vulnerable to SQL injection, allowing an attacker to interfere with the execution of SQL queries and access restricted areas without valid credentials.

***Conclusion** –

Skills Gained:

- Learned how SQL injection works and how input manipulation can alter SQL query logic.
- Practiced identifying weak input validation mechanisms in login forms.
- Gained insight into how authentication bypass via SQLi can be exploited in real-world web applications.

Challenges Faced:

- Faced initial difficulty in finding the right injection point and payload that would work on the login page.
- Some responses were ambiguous, requiring careful observation of server behavior after injection attempts.
- Needed to try multiple payload variations to confirm vulnerability.

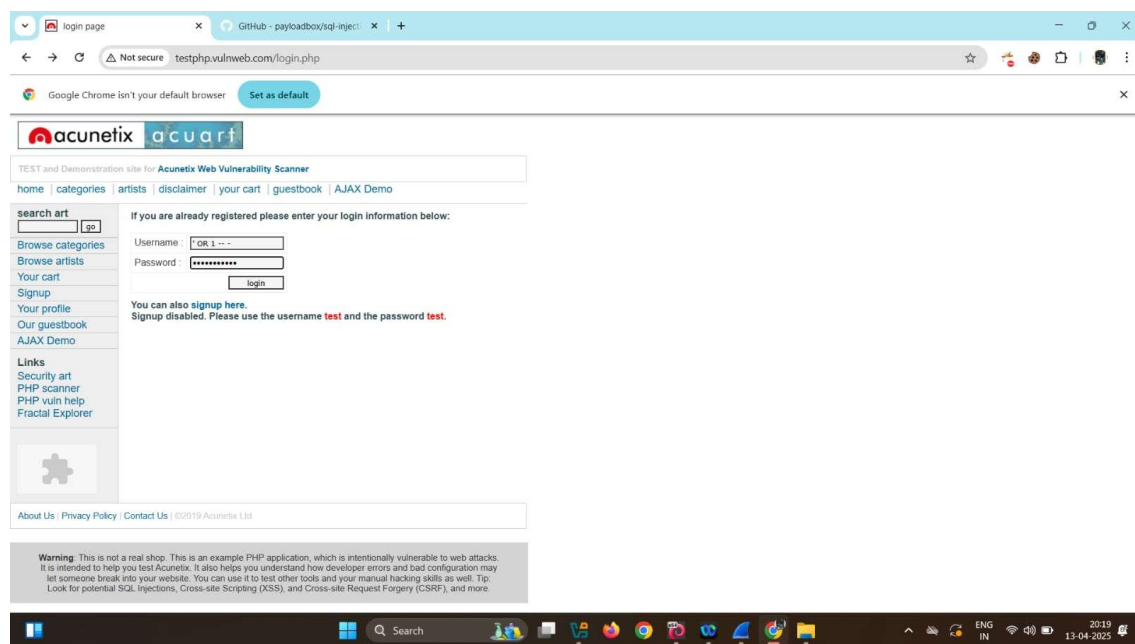
Improvements:

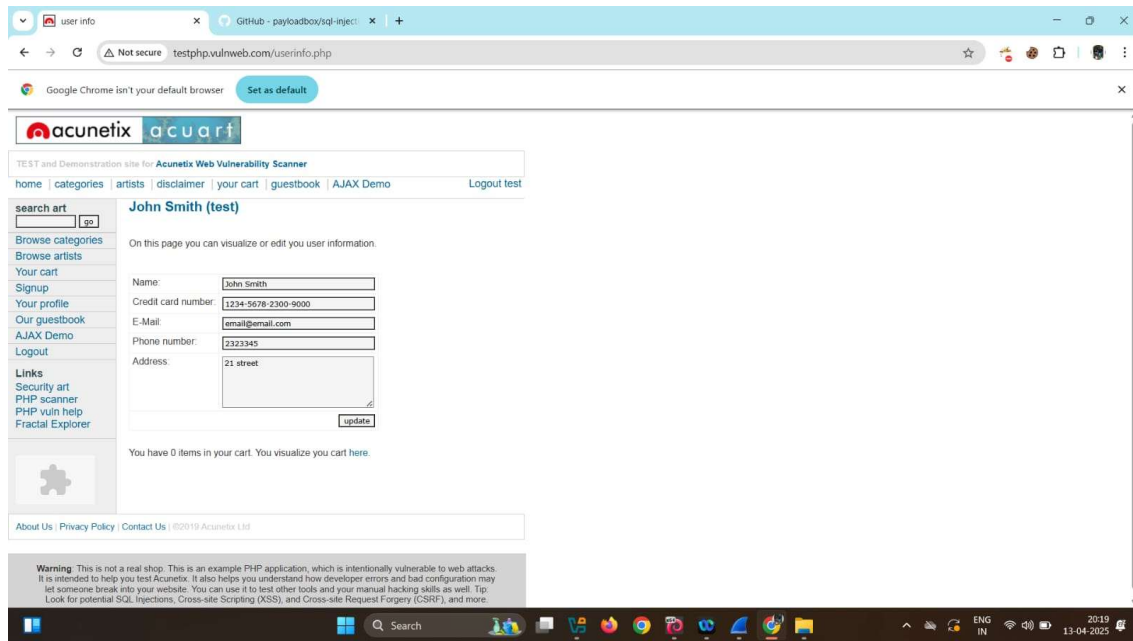
- Developed a better understanding of how to craft effective SQL injection payloads.
- Improved analysis of server response behavior to detect SQLi success (e.g., changes in page access or error messages).
- Became more familiar with the logic behind query structure and how it can be exploited when improperly coded.

*Screenshot -

Title: SQL Injection Authentication Bypass on `testphp.vulnweb.com`

The screenshot shows a successful SQL injection attack carried out on the login form of `http://testphp.vulnweb.com/`, where the payload `' OR '1'='1 --` was used to bypass user authentication. After submitting the form with this input, the application granted access without requiring valid login credentials. This demonstrates a classic case of **SQL injection vulnerability**, where unsanitized inputs allow attackers to manipulate backend queries. The exercise illustrates the importance of secure coding practices, such as input sanitization and the use of prepared statements to prevent such attacks.





Task 5 - Injected malicious JavaScript payloads in input fields (Xss – cross site scripting)

***Outcomes –**

Summary :

To evaluate the website <http://testphp.vulnweb.com/> for reflected XSS vulnerabilities, I tested input fields—particularly the search bar—by injecting JavaScript payloads and observing the browser's response.

Using a basic XSS payload:

`<script>alert(1)</script>`

I injected it into the search query parameter. Upon submission, the JavaScript was executed immediately in the browser, confirming the website is vulnerable to reflected XSS. This type of vulnerability occurs when untrusted user input is immediately included in the web response without proper sanitization or encoding.

The alert box popping up on the site demonstrated that arbitrary scripts could be executed in the victim's browser, making it possible for attackers to steal session cookies, perform phishing, or manipulate page content.

***Conclusion –**

Skills Gained:

- Gained hands-on experience identifying and exploiting reflected XSS vulnerabilities.
- Learned how unsanitized inputs in URL parameters can be used to inject and execute scripts.

- Understood the impact of client-side attacks and how they can be used to compromise user data or hijack sessions

Challenges Faced:

- Initially, it was unclear which parameters on the website were vulnerable to XSS, requiring a trial-and-error approach.
- Some inputs were sanitized or returned encoded, making it necessary to try multiple payload formats.
- Understanding how the server processed input and reflected it back into the HTML structure required close observation.

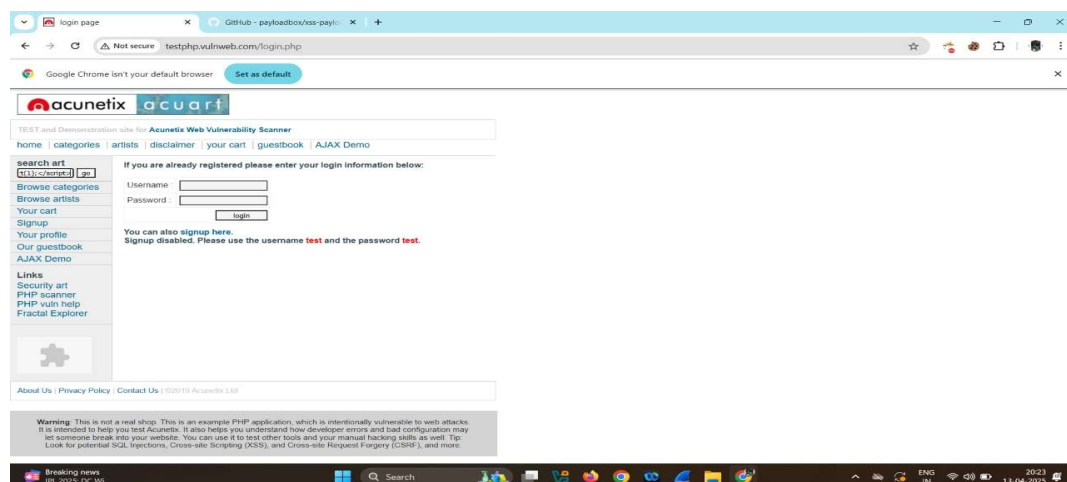
Improvements:

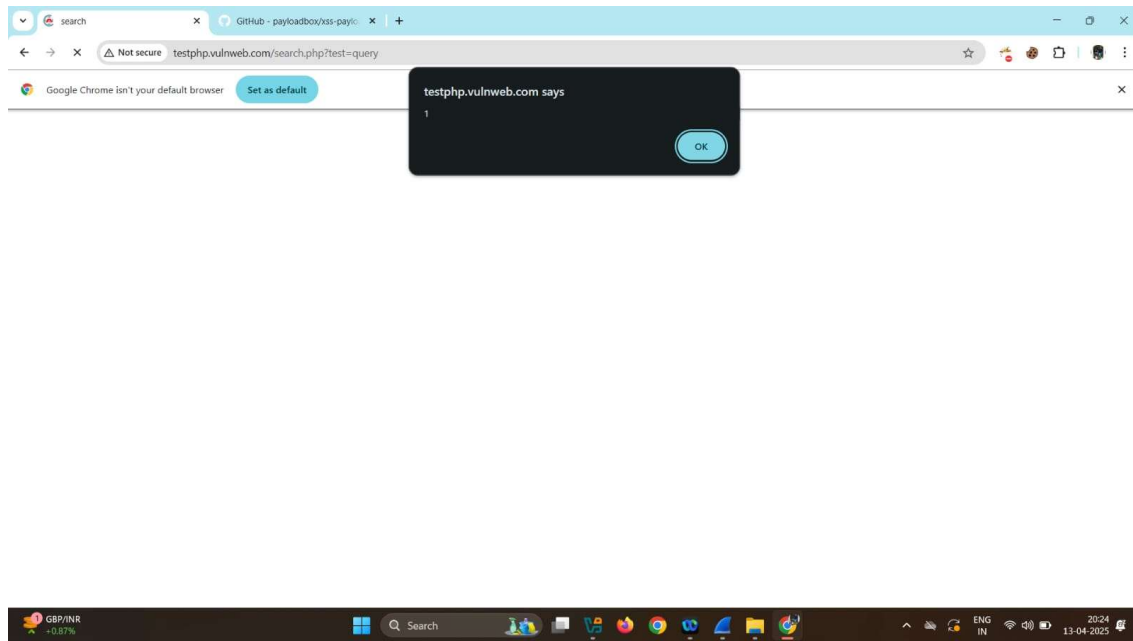
- Improved ability to craft payloads for different contexts (HTML, script, attribute).
- Became more efficient at identifying injection points in URL parameters and form inputs.
- Gained deeper insight into browser behavior when executing injected scripts, which will help in testing for stored or DOM-based XSS in future.

*Screenshot -

Title: Reflected XSS Exploitation on <http://testphp.vulnweb.com/>

This screenshot shows the successful execution of a **reflected Cross-Site Scripting (XSS)** payload injected through the search parameter of <http://testphp.vulnweb.com/>. The payload `<script>alert('XSS')</script>` triggered an alert pop-up in the browser, confirming that the website reflects user input back into the page without proper sanitization. This vulnerability allows attackers to execute arbitrary scripts in a victim's browser session and demonstrates the critical need for **input validation**, **output encoding**, and the use of **Content Security Policies (CSP)** in secure web development.





ETHICAL HACKING PROJECT

Port Scanner - Working Process and Key Functions of them.

How It Works:

1. Input Collection – User enters a target IP address and a port range.
2. Validation – The script checks if the IP and ports are valid.
3. Port Scanning – Uses Python's `socket` module to attempt TCP connections to each port:
 - If connection is successful, the port is marked as open.
 - If it fails or times out, the port is considered closed.
4. Output – Displays a list of open ports to the user.

Key Functions:

- `input()` – Get user inputs.
- `scan\_port()` – Tries to connect to a specific port using `socket.connect\_ex()`.
- `socket.setdefaulttimeout()` – Limits wait time per connection.
- `try/except` – Handles invalid inputs or errors.
- `print()` – Shows open ports.

Port 80 Open Explanation :

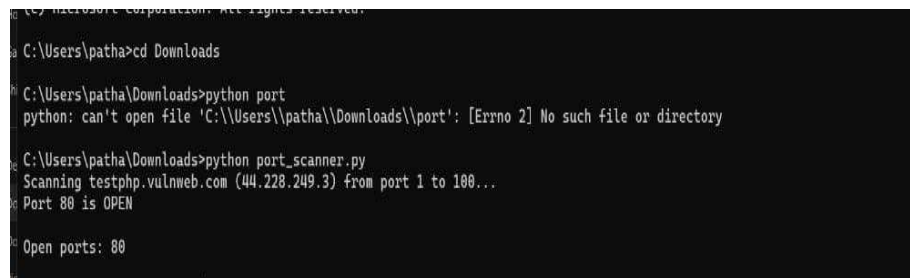
Port 80 is the default port for HTTP. If it's open, a web server is likely running on the target IP, accessible via a browser using `http://testphp.vulnweb.com`.

*Screenshot -

Result of Basic Python Port Scanner :

Description:

The screenshot shows the execution of a basic Python port scanner in command prompt. It displays the list of open ports discovered on a target IP address. Notably, port 80 is open, indicating the presence of a running HTTP web server.



```
(C) Microsoft Corporation. All rights reserved.  
C:\Users\patha>cd Downloads  
C:\Users\patha\Downloads>python port  
python: can't open file 'C:\\Users\\patha\\Downloads\\port': [Errno 2] No such file or directory  
C:\Users\patha\Downloads>python port_scanner.py  
Scanning testphp.vulnweb.com (44.228.249.3) from port 1 to 100...  
Port 80 is OPEN  
Open ports: 80
```

Learning Outcomes -

During the internship, I have gained significant hands-on experience and developed key cybersecurity skills:

- **Technical Proficiency:**
 - Mastered tools like **Nmap** for open port scanning, **Dirsearch** for brute-force directory enumeration, and **Wireshark** for traffic analysis.
 - Performed real-world penetration testing using **SQL Injection** and **Cross-Site Scripting (XSS)** techniques.
 - Built and executed a **Python-based port scanner**, strengthening my scripting and networking knowledge.

Challenges and Solutions -

I have faced several technical and practical hurdles during the internship:

- **Challenge:** Difficulty understanding scan types and interpreting filtered results in Nmap.
 - **Solution:** Referred to official documentation and practiced in a virtual lab environment.

- **Challenge:** Raw Dirsearch outputs were overwhelming and required manual inspection.
 - **Solution:** Enhanced my command-line skills and used filters to improve result relevance.
- **Challenge:** Identifying the right packets in Wireshark amidst noisy traffic.
 - **Solution:** Learned to apply effective display filters and refined my packet analysis skills.
- **Challenge:** SQL injection and XSS exploitation required trial and error due to unpredictable responses.
 - **Solution:** Improved my payload crafting ability and learned to interpret server/client behavior accurately.

Conclusion -

My internship at Hack Secure provided me with an immersive experience in the field of cybersecurity. Through practical exposure to vulnerability assessment, penetration testing, and secure coding analysis, I've developed a strong foundation in ethical hacking. Despite facing technical challenges, my proactive approach and continuous learning attitude helped me overcome them and grow as a cybersecurity professional. The skills that I have acquired not only enhanced my technical competence but also prepared me for real-world security scenarios.

Acknowledgment -

I express my sincere gratitude to Hack Secure, especially my mentor, **Mr.Nishant Prajapati**, and assistant mentor **Mr. Aman Pandey**, for their guidance and support throughout my internship. I also thank **Amrita Vishwa Vidyapeetham** for providing this internship opportunity, which has been crucial in my personal and professional development. This report encapsulates the essence of my internship experience, highlighting the integration of academic knowledge with practical skills in a professional setting. It reflects my journey of learning, growth, and development in the field of cybersecurity